Universidad del Valle de Guatemala

Facultad de Ingeniería

Programación Orientada a Objetos

Catedrático/a:  Erick Marroquín




# Ejercicio 6 - Polimorfismo vía interfaces

Nombre: Javier Alejandro Apen Solis

Carné: 26864

03/11/2026

## Requisitos y criterios de aceptación

Actor principal: administrador del sistema. Los proveedores simulados son objetos del modelo; no interviene una red externa en este prototipo.

| ID | Requisito | Evidencia de aceptación |
|---|---|---|
| RF-01 | Cargar al menos diez APIs diversas al iniciar. | CargaInicial crea diez objetos: seis pagos, dos logística, una consulta y una FEL. |
| RF-02 | Listar integraciones activas y estado de conexión. | Opción 1 muestra id, nombre, tipo, conexión y latencia; filtra las inactivas. |
| RF-03 | Buscar por identificador y consultar configuración | Opción 2 muestra endpoint, timeout, estado, latencia y datos específicos. Un id desconocido produce un mensaje. |
| RF-04 | Ordenar por latencia promedio. | Opción 3 ordena ascendentemente la misma lista; desempata por id y conserva ese orden. |
| RF-05 | Consultar estado, rastreo o cotización. | Opción 4 delega al comportamiento concreto; logística informa tarifa y rastreo. |
| RF-06 | Ejecutar cobros y reintegros. | Opciones 5 y 6 modifican saldo de pasarelas y validan monto/saldo. |
| RF-07 | Ignorar operaciones incompatibles sin colapsar. | Reintegrar a LOG-01 devuelve IGNORADO; la siguiente acción se ejecuta normalmente. |
| RF-08 | Generar documentos. | Opción 7 genera guía o XML FEL demostrativo; un proveedor sin capacidad devuelve IGNORADO. |
| RF-09 | Permitir auditoría. | Opción 8 presenta contador y último evento de la sesión, incluidos ERROR e IGNORADO. |
| RF-10 | Añadir proveedores sin cambiar el motor. | Registrar una nueva instancia logística habilita sus |

| | | |
|---|---|---|
| | | operaciones con el mismo controlador y motor. |
| RF-11 | Mantener un menú usable y salida limpia. | Opción 0 y fin de entrada cierran; entradas inválidas muestran mensajes y permiten continuar. |

## Diseño orientado a objetos

| Capa MVC | Clases y responsabilidad |
|---|---|
| Modelo | IntegracionAPI y sus subclases contienen estado y reglas. CatalogoIntegraciones administra la colección. MotorOperaciones trabaja mediante interfaces. CargaInicial aporta datos. |
| Controlador | ControladorIntegraciones recibe id y argumentos, valida disponibilidad, invoca el motor o la consulta y registra el resultado. No imprime ni importa Swing. |
| Vista y driver | Principal contiene main, Scanner, el menú, el formato de presentación y la ventana Swing. Es el único archivo que usa System.out. |

## Herencia y composición

La clase abstracta IntegracionAPI evita duplicar identidad, configuración, estado, latencia y auditoría. PasarelaPago, ProveedorLogistico, ServicioConsulta y CertificadorFEL extienden esa clase. El catálogo contiene referencias de tipo IntegracionAPI a objetos de cualquiera de estas clases. El controlador utiliza un catálogo y un motor recibidos por constructor.

## Interfaces con parámetros polimórficos

MotorOperaciones no recibe PasarelaPago ni ProveedorLogistico: recibe Transaccional, Documentable o Auditable. Puede atender incluso un objeto que implemente el contrato sin heredar de IntegracionAPI. Las pruebas incluyen ese caso para comprobar que la dependencia es realmente sobre la interfaz

## Contrato de operación neutra y sustitución

Transaccional y Documentable definen métodos default que devuelven IGNORADO. El contrato significa “intentar atender esta operación y comunicar su resultado”, no “garantizar que cualquier proveedor puede cobrar”. Por ello, un objeto logístico puede sustituir al contrato sin lanzar una excepción por una capacidad ausente. Las pasarelas sobrescriben los métodos financieros; logística y FEL sobrescriben la generación documental. Esta decisión es deliberada para el requisito de ignorar solicitudes incompatibles. Los clientes del motor siguen dependiendo de contratos pequeños por comportamiento. No se usan comprobaciones de clase ni conversiones de tipo para escoger capacidades.

## Responsabilidades y extensibilidad

El motor permanece cerrado a modificaciones por la incorporación de proveedores compatibles. Las variaciones se agregan mediante configuración o subclases. El constructor de cada entidad valida errores de configuración del programador; los casos esperados del usuario y del negocio se comunican mediante ResultadoOperacion.

## Clases y contratos

| Clase | Estado o contrato | Definición |
|---|---|---|
| IntegracionAPI | id, nombre, endpoint y timeout inmutables; activa, conexión, latencia y auditoría. | Base común; consulta, validación, identidad, comparación y escape XML protegido. |
| PasarelaPago | MedioPago y saldo BigDecimal. | Cobrar y reintegrar; no genera documentos |
| ProveedorLogistico | Tarifa base BigDecimal. | Rastreo, cotización y guía XML. |
| ServicioConsulta | Estado heredado. | Solo rastreo; no mueve dinero ni crea documentos. |
| CertificadorFEL | Estado heredado. | XML de demostración; no certifica fiscalmente. |
| CatalogoIntegraciones | ArrayList <IntegracionAPI>. | Agregar sin duplicados, buscar, ordenar e iterar. |
| MotorOperaciones | Sin estado mutable | Despacho mediante interfaces; métodos procesar sobrecargados. |
| CargaInicial | Sin instancias; método estático. | Crear catálogo con diez proveedores ficticios |
| ControladorIntegraciones | Referencias finales al catálogo y motor. | Coordinar solicitudes y registrar auditoría. |
| Principal | Referencia final al controlador. | Pedir datos, convertir entradas, presentar resultados y controlar el menú. |

| | | |
|---|---|---|
| ResultadoOperacion | EstadoOperacion, mensaje y contenido inmutables. | Respuesta común; fábricas exito, ignorado y error. |
| Transaccional | cobrar y reintegrar reciben BigDecimal. | Respuesta neutra por defecto; ambas devuelven ResultadoOperacion. |
| Documentable | generarDocumento recibe String. | Contrato documental con resultado neutro por defecto. |
| Auditable | registrarEvento(String); consultarAuditoria(). | Registrar y consultar el resumen de sesión. |

## Casos de uso y respuestas del sistema

| Caso | Uso principal y alternativas. |
|---|---|
| CU-01 · Listar / ordenar | El administrador selecciona 1 o 3. El controlador entrega el Iterable; si se pidió ordenar, el catálogo aplica Comparable antes de mostrarlo. Principal presenta solo activas. Una colección vacía se muestra con total cero. |
| CU-02 · Buscar | El administrador introduce un id. El catálogo normaliza mayúsculas y espacios exteriores, busca linealmente y devuelve la entidad o null. La vista muestra configuración o “no se encontró”. |
| CU-03 · Cobrar / reintegrar | La vista convierte el monto. El controlador busca y valida disponibilidad. El motor recibe Transaccional y ejecuta el método seleccionado. La pasarela valida monto/saldo, modifica el saldo y retorna EXITO o ERROR. El controlador audita el resultado. |
| CU-04 · Reintegro incompatible | Con LOG-01 disponible, el motor invoca reintegrar. Se ejecuta el default de Transaccional, que retorna IGNORADO sin alterar el estado. El controlador registra el intento y la vista vuelve al menú. |
| CU-05 · Documento / consulta | La vista recibe una referencia. El controlador verifica la API y delega. La subclase valida la referencia; genera XML, informa rastreo/cotización o conserva el default IGNORADO. |
| CU-06 · Auditoría / salida | La opción 8 consulta contador y último evento sin alterar ese contador. La opción 0 o el fin del flujo de entrada termina el ciclo y cierra la sesión. |

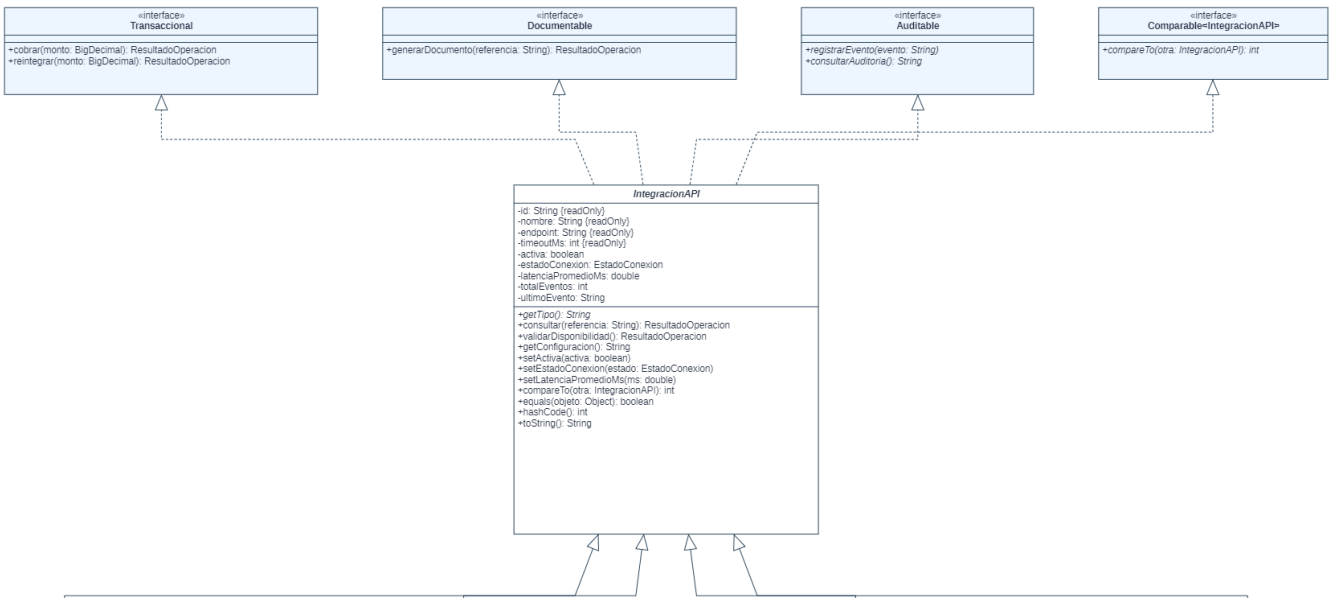
## Orden de validaciones

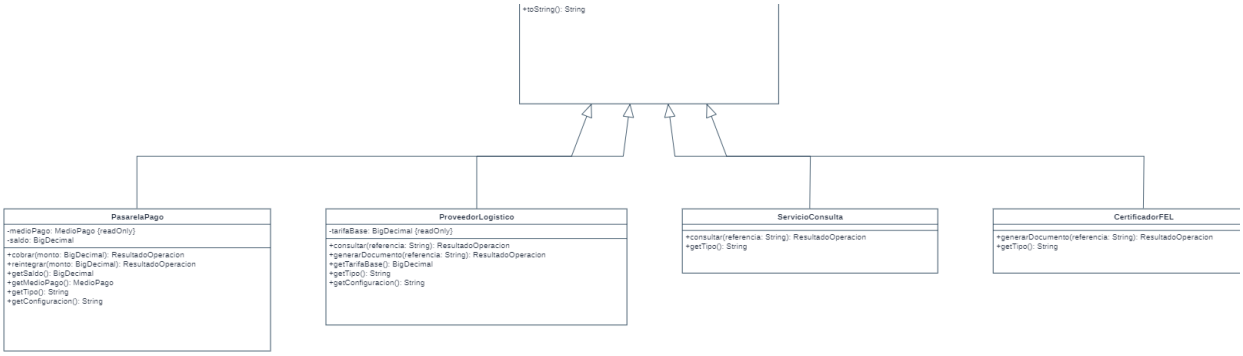
La sintaxis de entrada se valida en la vista. Luego el controlador verifica que el id exista y que la integración esté activa y disponible. Después se evalúan las reglas de la operación. Por tanto, LOG-02 desconectada devuelve ERROR de disponibilidad, mientras que LOG-01 conectada devuelve IGNORADO ante un reintegro. Ninguno de esos resultados detiene el menú.

## Dinero, referencias y auditoría

Se usa BigDecimal para evitar errores de representación binaria. Un monto debe ser positivo y tener hasta dos decimales. Un reintegro no puede superar el saldo neto de la pasarela en la sesión. Las referencias admiten de 1 a 80 caracteres tras quitar espacios exteriores. Los caracteres especiales se escapan antes de formar XML. La auditoría guarda un contador, fecha/hora local del equipo y último evento. Se registran consultas, cobros, reintegros y documentos, con sus estados de respuesta. Un id inexistente no puede asociarse a una entidad para auditar. No se conserva un historial ni hay persistencia.

Diseño






Enlace GitHub

https://github.com/MR-APEN/Ejercicio-6.-Polimorfismo-v-a-interfaces